

Universidad Torcuato Di Tella

Licenciatura en Tecnología Digital

Tecnología Digital V: Diseño de Algoritmos

Primer Semestre, 2026

TP2 — Logística centralizada de primera milla

Integrantes

Ariel Alzogaray Flores, Damian Cevallos Ortiz, Josefina Deserti Gotta

Fecha de entrega: 19 de junio de 2026

Índice

1. Introducción	2
2. Modelado	2
2.1. Planteo del problema	2
2.2. Representación de la solución	2
3. Algoritmos implementados	3
3.1. Heurísticas constructivas	3
3.1.1. Depósito a menor distancia	3
3.1.2. Depósito a menor distancia (con vendedores ordenados por promedio de distancia)	3
3.1.3. Vendedor a menor distancia	4
3.1.4. Asignación sujeta a ratio distancia-demanda	5
3.2. Operadores de búsqueda local	5
3.2.1. <i>Swap</i>	6
3.2.2. <i>Relocate</i>	6
3.3. Metaheurística: Grasp	7
4. Experimentación y análisis del caso real	8
4.1. Metodología	9
4.2. Experimento 1: Comparación de tiempos de ejecución entre las heurísticas constructivas	9
4.3. Experimento 2: Impacto de la heurística constructiva elegida para los operadores de búsqueda locales	9
4.4. Experimento 3: Tuning de α en GRASP	10
5. Uso de herramientas de AI	10
6. Conclusiones	10

1. Introducción

A raíz del auge de la compra y venta en línea de artículos y su consolidación en el mercado, la logística en términos de recepción y entrega de paquetes se complejizó de modo tal que la gran mayoría de empresas destina una gigantesca cantidad de recursos a mejorar su eficiencia. Uno de los problemas que se deriva de este avance llega antes del momento de entregar el paquete al comprador. Para que esto sea posible, la infraestructura de recepción y almacenamiento de paquetes debe ser óptima para minimizar los costos operativos, y es de especial interés que cada vendedor sea asignado a un depósito que sea capaz de albergar su mercadería a la vez que minimiza los costos de traslado.

Uno de los enfoques más populares para lograr esto fue el tecnológico, es decir, se optó por el desarrollo de *software* especializado para encontrar soluciones eficientes. En lo que al diseño de algoritmos respecta, este problema no sólo es interesante por su relevancia en la actualidad sino también por su complejidad: el Problema de Asignación Generaliza (GAP) que se utiliza para modelar este contexto es NP-difícil, por lo que no existen algoritmos que encuentren soluciones óptimas en tiempo polinomial sin recurrir a una restricción del problema. El objetivo de este trabajo es desarrollar heurísticas más rápidas que los algoritmos exactos, a costa de sacrificar una cantidad aceptable de precisión.

2. Modelado

2.1. Planteo del problema

Se tienen los siguientes parámetros a considerar:

1. Un entero n que indica la cantidad de vendedores que deben almacenar sus paquetes.
2. Un entero m que indica la cantidad de depósitos disponibles en el mapa.
3. Un vector de capacidades enteras Ca con $|Ca| = m$ tal que Ca_i representa la capacidad del i -ésimo depósito.
4. Una matriz de costos enteros $Co^{m \times n}$ tal que Co_{ij} indica el costo que tiene para el vendedor j depositar sus paquetes en el depósito i .
5. Una matriz de demandas enteras $D^{m \times n}$ tal que D_{ij} indica la capacidad que demanda el vendedor j si elige almacenar sus paquetes en el depósito i .
6. Un entero $cmax$ que indica el costo máximo encontrado entre todos los pares posibles de vendedores y depósitos. Este número se tiene en cuenta a la hora de penalizar los vendedores no asignados. Dicho costo adicional está dado por $3 \times cmax \times \text{vendedores_sin_asignar}$.

Dados estos parámetros, el objetivo es asignar cada vendedor j a exactamente un depósito i respetando las capacidades de estos, a la vez que se minimiza el costo total de la asignación.

2.2. Representación de la solución

Una solución en el marco de este trabajo es representada por una matriz de enteros $S^{m \times n}$, de forma tal que cada vector S_i contiene todos los vendedores asignados al i -ésimo depósito. Dado que en esta investigación se ha optado por desarrollar algoritmos aproximados, es importante tener en cuenta que rara vez será posible hallar el mínimo global de todo el conjunto de asignaciones

posibles. Sin embargo, resulta interesante observar qué tan cerca se puede llegar dependiendo del criterio que se aplique en cada caso.

3. Algoritmos implementados

Con el objetivo de diversificar la perspectiva que se tiene sobre el problema y de explorar los estándares de calidad que se pueden lograr, se han propuesto cuatro heurísticas constructivas, dos operadores de búsqueda local y una metaheurística.

Cabe aclarar que durante el desarrollo de estos algoritmos también desarrollamos heurísticas alternativas cuyos desempeños no fueron lo suficientemente satisfactorias como para ser incluidas.

3.1. Heurísticas constructivas

Esta familia de algoritmos construye soluciones factibles paso a paso sin volver atrás sobre las decisiones ya tomadas. La elección depende fuertemente del criterio utilizado.

3.1.1. Depósito a menor distancia

Como su nombre lo indica, aquí se aplica un criterio *greedy* desde la mirada del vendedor. Se elegirá el depósito más cercano para cada vendedor en orden secuencial, con la sutileza de que algunos de ellos podrán quedar sin asignar.

Algoritmo 1 HC1: Depósito a menor distancia

```

1: función HC1( $Co, Ca, D, m, n, cmax$ )
2:   Inicializar  $S[i] \leftarrow \emptyset \forall i$  y  $cap\_res \leftarrow Ca$   $\triangleright \Theta(m)$ 
3:   para cada vendedor  $j$  hacer  $\triangleright \mathcal{O}(n)$ 
4:      $i^* \leftarrow$  depósito asignable de menor distancia a  $j$   $\triangleright \mathcal{O}(m)$ 
5:     si  $\exists i^*$  entonces
6:        $S[i^*] \leftarrow S[i^*] \cup \{j\}$ , actualizar  $cap\_res[i^*]$   $\triangleright \Theta(1)$ 
7:     sino
8:       Registrar  $j$  como vendedor sin asignar
9:     fin si
10:  fin para
11:   $costo\_total \leftarrow costo\_total + 3 \times cmax \times |vendedores \text{ sin asignar}|$ 
12:  retornar ( $S$ ,  $costo\_total$ )
13: fin función
```

Complejidad temporal: $\mathcal{O}(n) \times \mathcal{O}(m) = \boxed{\mathcal{O}(n \times m)}$.

Complejidad espacial: $\Theta(m) + n \times \Theta(1) = \Theta(m) + \Theta(n) = \boxed{\Theta(m + n)}$

3.1.2. Depósito a menor distancia (con vendedores ordenados por promedio de distancia)

Se mantiene la perspectiva del algoritmo anterior. Sin embargo, en este enfoque se calcula para cada vendedor su costo promedio y se sigue un orden de asignación creciente en función de estos. En otras palabras, aquellos vendedores con menor costo promedio serán asignados antes,

mientras que quienes presenten un promedio más elevado serán asignados hacia el final de la ejecución.

Algoritmo 2 HC2: Depósito a menor distancia + promedios

```

1: función HC2( $Co, Ca, D, m, n, cmax$ )
2:   Inicializar  $S[i] \leftarrow \emptyset \forall i$  y  $cap\_res \leftarrow Ca$   $\triangleright \Theta(m)$ 
3:    $prom[j] \leftarrow$  costo promedio de  $j$  sobre los depósitos que pueden asignarlo  $\forall j \triangleright \mathcal{O}(n \times m)$ 
4:   mientras  $\exists$  vendedores sin asignar hacer  $\triangleright \mathcal{O}(n)$ 
5:      $j^* \leftarrow$  vendedor no asignado con menor costo promedio  $\triangleright \mathcal{O}(n)$ 
6:      $i^* \leftarrow$  depósito asignable a  $j^*$  de menor costo  $\triangleright \mathcal{O}(m)$ 
7:     si  $\exists i^*$  entonces
8:        $S[i^*] \leftarrow S[i^*] \cup \{j^*\}$ , actualizar  $cap\_res[i^*]$   $\triangleright \Theta(1)$ 
9:     sino
10:      Registrar  $j^*$  como vendedor sin asignar
11:   fin si
12:   Actualizar  $prom[j]$  para los vendedores restantes  $\triangleright \mathcal{O}(n \times m)$ 
13: fin mientras
14:  $costo\_total \leftarrow costo\_total + 3 \times cmax \times |vendedores \text{ sin asignar}|$ 
15: retornar ( $S$ ,  $costo\_total$ )
16: fin función

```

Complejidad temporal: $\mathcal{O}(n \times m) + \mathcal{O}(n) \times [\mathcal{O}(n) + \mathcal{O}(m) + \mathcal{O}(n \times m)] = \mathcal{O}(n \times m) + \mathcal{O}(n) \times \mathcal{O}(n \times m) = \mathcal{O}(n \times m) + \mathcal{O}(n^2 \times m) = \boxed{\mathcal{O}(n^2 \times m)}$.

Complejidad espacial: $\Theta(m) + n \times \Theta(1) = \Theta(m) + \Theta(n) = \boxed{\Theta(m + n)}$

3.1.3. Vendedor a menor distancia

En contraposición al primer algoritmo, esta vez el criterio *greedy* se aplica desde la perspectiva del depósito. Cabe señalar que se itera una sola vez sobre todos los depósitos, pasando al siguiente sólo cuando no sea posible asignar nada más al actual (cuando se llena). Esta aclaración será relevante en la próxima heurística.

Algoritmo 3 HC3: Vendedor a menor distancia

```

1: función HC3( $Co, Ca, D, m, n, cmax$ )
2:   Inicializar  $S[i] \leftarrow \emptyset \forall i$ ,  $cap\_res \leftarrow Ca$  y todos los vendedores disponibles  $\triangleright \Theta(m + n)$ 
3:   para cada depósito  $i$  hacer
4:     mientras  $\exists$  vendedor disponible asignable a  $i$  hacer  $\triangleright \mathcal{O}(n + m)$ 
5:        $j^* \leftarrow$  vendedor disponible de menor costo asignable a  $i$   $\triangleright \mathcal{O}(n)$ 
6:        $S[i] \leftarrow S[i] \cup \{j^*\}$ , actualizar  $cap\_res[i]$   $\triangleright \Theta(1)$ 
7:       Marcar  $j^*$  como no disponible
8:     fin mientras
9:   fin para
10:  Registrar como vendedores sin asignar a los que quedaron disponibles  $\triangleright \mathcal{O}(n)$ 
11:   $costo\_total \leftarrow costo\_total + 3 \times cmax \times |vendedores \text{ sin asignar}|$ 
12:  retornar ( $S$ ,  $costo\_total$ )
13: fin función

```

Complejidad temporal: $\mathcal{O}(m + n) \times \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(m \times n + n^2) + \mathcal{O}(n) = \boxed{\mathcal{O}(m \times n + n^2)}$.

Complejidad espacial: $\Theta(m+n) + n \times \Theta(1) = \Theta(m+n) + \Theta(n) = \boxed{\Theta(m+n)}$

3.1.4. Asignación sujeta a ratio distancia-demanda

El ratio distancia-demanda ofrece una intuición de qué tan eficiente es una posible asignación, independientemente del resto de la solución. Cuanto más pequeño sea el ratio, mejor será la elección. Se puede pensar de la siguiente forma:

- Cuando el ratio es pequeño, el costo es bajo en comparación a la demanda. Significa que los gastos de traslado se consideran justificados dada la magnitud del espacio demandado.
- Cuando el ratio es grande, el costo es muy grande en comparación a la demanda. Esto indica que el sacrificio necesario para concretar el traslado no vale la pena para el poco espacio que se requiere del depósito.

De esta manera, las asignaciones se realizan en orden de menor a mayor ratio siempre y cuando sea factible.

Algoritmo 4 HC4: Ratio distancia-demanda

```

1: función HC4( $Co, Ca, D, m, n, cmax$ )
2:   Inicializar  $S[i] \leftarrow \emptyset \forall i$ ,  $cap\_res \leftarrow Ca$  y todos los vendedores disponibles  $\triangleright \Theta(m+n)$ 
3:    $R_{ij} \leftarrow Co_{ij}/D_{ij} \forall i, j$   $\triangleright \Theta(m \times n), \mathcal{O}(m \times n)$ 
4:    $orden \leftarrow$  pares  $(i, j)$  ordenados de menor a mayor  $R_{ij}$   $\triangleright \Theta(m \times n), \mathcal{O}(m^2 \times n^2)$ 
5:   para cada par  $(i, j) \in orden$  hacer  $\triangleright \mathcal{O}(m \times n)$ 
6:     si  $j$  disponible y asignable a  $i$  entonces
7:        $S[i] \leftarrow S[i] \cup \{j\}$ , actualizar  $cap\_res[i]$   $\triangleright \Theta(1)$ 
8:       Marcar  $j$  como no disponible
9:     fin si
10:  fin para
11:  Registrar como vendedores sin asignar a los que quedaron disponibles
12:   $costo\_total \leftarrow costo\_total + 3 \times cmax \times |vendedores \text{ sin asignar}|$ 
13:  retornar ( $S$ ,  $costo\_total$ )
14: fin función

```

Complejidad temporal: $\mathcal{O}(m \times n) + \mathcal{O}(m^2 \times n^2) + \mathcal{O}(m \times n) = \boxed{\mathcal{O}(m^2 \times n^2)}$.

Complejidad espacial: $\Theta(m+n) + \Theta(m \times n) + \Theta(m \times n) + n \times \Theta(1) = 2 \times \Theta(m \times n) = \boxed{\Theta(m \times n)}$

3.2. Operadores de búsqueda local

Se ha optado por implementar dos operadores clásicos para GAP: *swap* y *relocate*. En ambos casos, se utiliza el criterio de aceptación *first-improvement*. Bajo este esquema, se busca la primera mejora y se ignoran todas las demás, en contraposición a *best-improvement*, donde se busca la mejora óptima.

3.2.1. Swap

Algoritmo 5 Operador *swap*

```

1: función SWAP( $Co, Ca, D, m, n, cmax$ )
2:   ( $S, costo\_total$ )  $\leftarrow$  HC1( $Co, Ca, D, m, n, cmax$ )  $\triangleright \mathcal{O}(nm), \Theta(n + m)$ 
3:   mientras  $\exists$  mejora hacer
4:     mejora  $\leftarrow$  falso
5:     para todo  $j_1 \in S[i_1], j_2 \in S[i_2]$  con  $i_1 \neq i_2$  mientras no haya mejora hacer  $\triangleright \mathcal{O}(n^3)$ 
6:       si intercambiar  $j_1$  y  $j_2$  reduce el costo y es factible entonces
7:          $S[i_1] \leftarrow (S[i_1] \setminus \{j_1\}) \cup \{j_2\}$ 
8:          $S[i_2] \leftarrow (S[i_2] \setminus \{j_2\}) \cup \{j_1\}$ 
9:         Actualizar  $cap\_res$  y  $costo\_total$ 
10:        mejora  $\leftarrow$  verdadero
11:      fin si
12:    fin para
13:  fin mientras
14:  retornar ( $S, costo\_total$ )
15: fin función

```

Complejidad temporal: $\mathcal{O}(nm) + \mathcal{O}(n^3) = \boxed{\mathcal{O}(n^3)}$.

Complejidad espacial: $\boxed{\Theta(n + m)}$

3.2.2. Relocate

Algoritmo 6 Operador *relocate*

```

1: función RELOCATE( $Co, Ca, D, m, n, cmax$ )
2:   ( $S, costo\_total$ )  $\leftarrow$  HC2( $Co, Ca, D, m, n, cmax$ )  $\triangleright \mathcal{O}(nm), \Theta(n + m)$ 
3:   mientras  $\exists$  mejora hacer
4:     mejora  $\leftarrow$  falso
5:     para cada vendedor asignado  $j \in S[i_1]$  y cada depósito  $i_2 \neq i_1$ , mientras no haya  $\triangleright \mathcal{O}(n^2m)$ 
6:       si reubicar  $j$  de  $i_1$  a  $i_2$  reduce el costo y es factible entonces
7:          $S[i_1] \leftarrow S[i_1] \setminus \{j\}$ 
8:          $S[i_2] \leftarrow S[i_2] \cup \{j\}$ 
9:         Actualizar  $cap\_res$  y  $costo\_total$ 
10:        mejora  $\leftarrow$  verdadero
11:      fin si
12:    fin para
13:  fin mientras
14:  retornar ( $S, costo\_total$ )
15: fin función

```

Complejidad temporal: $\mathcal{O}(nm) + \mathcal{O}(n^2m) = \boxed{\mathcal{O}(n^2m)}$.

Complejidad espacial: $\boxed{\Theta(n + m)}$

3.3. Metaheurística: Grasp

Para el caso de la implementación de una metaheurística, hemos optado por implementar la metaheurística GRASP (*Greedy Randomized Adaptive Search Procedure*). En cada iteración se construye una solución factible mediante un greedy aleatorizado controlado por el parámetro $\alpha \in [0, 1]$, y luego se la mejora aplicando los operadores *relocate* y *swap* hasta alcanzar un óptimo local. Se conserva la mejor solución hallada entre todas las iteraciones.

Algoritmo 7 Metaheurística *GRASP*

```

1: función GRASP( $Co, Ca, D, m, n, cmax, n\_iter, \alpha$ )
2:   costo_min  $\leftarrow \infty$ 
3:    $S^* \leftarrow \emptyset$ 
4:   para  $k \leftarrow 1$  hasta  $n\_iter$  hacer  $\triangleright \mathcal{O}(n\_iter \cdot (nm + n^3))$ 
5:      $(S, cap\_res, costo\_total) \leftarrow \text{CONSTRUCTIVOGRASP}(Co, Ca, D, m, n, cmax, \alpha)$ 
6:      $(S, costo\_total) \leftarrow \text{BUSQUEDALOCALGRASP}(Co, D, m, n, S, cap\_res, costo\_total)$ 
7:     si  $costo\_total < costo\_min$  entonces
8:        $S^* \leftarrow S$ 
9:        $costo\_min \leftarrow costo\_total$ 
10:    fin si
11:  fin para
12:  retornar  $(S^*, costo\_min)$ 
13: fin función

```

Algoritmo 8 Construcción golosa aleatorizada

```

1: función CONSTRUCTIVOGRASP( $Co, Ca, D, m, n, cmax, \alpha$ )
2:    $S \leftarrow \{\emptyset, \dots, \emptyset\}$   $\triangleright m$  depósitos vacíos
3:    $cap\_res \leftarrow Ca$ 
4:    $costo\_total \leftarrow 0$ 
5:    $sin\_asignar \leftarrow 0$ 
6:    $J \leftarrow$  vendedores en orden aleatorio  $\triangleright \Theta(n)$ 
7:   para cada  $j \in J$  hacer  $\triangleright \mathcal{O}(nm)$ 
8:      $I_j \leftarrow \{i : D[i][j] \leq cap\_res[i]\}$   $\triangleright$  depósitos factibles para  $j$ 
9:     si  $I_j \neq \emptyset$  entonces
10:       $costo\_min \leftarrow \min_{i \in I_j} Co[i][j]$ 
11:       $costo\_max \leftarrow \max_{i \in I_j} Co[i][j]$ 
12:       $rango \leftarrow costo\_min + \alpha \cdot (costo\_max - costo\_min)$ 
13:       $LRC \leftarrow \{i \in I_j : Co[i][j] \leq rango\}$   $\triangleright$  lista restringida de candidatos
14:       $i^* \leftarrow$  elegir aleatoriamente (uniforme) de LRC
15:       $S[i^*] \leftarrow S[i^*] \cup \{j\}$ 
16:      Actualizar  $cap\_res[i^*]$  y  $costo\_total$ 
17:    sino
18:       $sin\_asignar \leftarrow sin\_asignar + 1$ 
19:    fin si
20:  fin para
21:   $costo\_total \leftarrow costo\_total + 3 \cdot cmax \cdot sin\_asignar$   $\triangleright$  penalización
22:  retornar  $(S, cap\_res, costo\_total)$ 
23: fin función

```

Algoritmo 9 Búsqueda local combinada (*relocate + swap*)

```

1: función BUSQUEDALOCALGRASP( $Co, D, m, n, S, \text{cap\_res}, \text{costo\_total}$ )
2:   mientras  $\exists$  mejora hacer
3:     mejora  $\leftarrow$  falso
4:     para todo  $j \in \{1, \dots, n\}$ ,  $i \neq$  depósito actual de  $j$  hacer  $\triangleright \mathcal{O}(nm)$ 
5:       si reubicar  $j$  en  $i$  reduce el costo y es factible entonces
6:         Quitar  $j$  de su depósito actual y agregarlo a  $S[i]$ 
7:         Actualizar  $\text{cap\_res}$  y  $\text{costo\_total}$ 
8:         mejora  $\leftarrow$  verdadero
9:       fin si
10:    fin para
11:    para todo  $j_1 \in S[i_1], j_2 \in S[i_2]$  con  $i_1 \neq i_2$  hacer  $\triangleright \mathcal{O}(n^3)$ 
12:      si intercambiar  $j_1$  y  $j_2$  reduce el costo y es factible entonces
13:         $S[i_1] \leftarrow (S[i_1] \setminus \{j_1\}) \cup \{j_2\}$ 
14:         $S[i_2] \leftarrow (S[i_2] \setminus \{j_2\}) \cup \{j_1\}$ 
15:        Actualizar  $\text{cap\_res}$  y  $\text{costo\_total}$ 
16:        mejora  $\leftarrow$  verdadero
17:      fin si
18:    fin para
19:  fin mientras
20:  retornar ( $S, \text{costo\_total}$ )
21: fin función

```

Complejidad temporal: $\underbrace{\mathcal{O}(nm)}_{\text{construcción}} + \underbrace{P \cdot (\mathcal{O}(n^2m) + \mathcal{O}(n^3))}_{\text{búsqueda local (relocate + swap)}} = \mathcal{O}(P \cdot n^2(n+m)) \xrightarrow{\times K \text{ iteraciones}}$

$$\boxed{\mathcal{O}(K \cdot P \cdot n^2(n+m))}$$

donde P denota la cantidad de pasadas hasta alcanzar un óptimo local en cada llamada a la búsqueda local.

Complejidad espacial: $\boxed{\Theta(n+m)}$

4. Experimentación y análisis del caso real

Los experimentos aquí dados tienen como objetivo investigar los siguientes fenómenos.

- Cómo afectan a la calidad de las soluciones los diferentes criterios desarrollados en las heurísticas.
- Cómo se alteran las soluciones de acuerdo a los parámetros utilizados.

De aquí en más notamos a las heurísticas bajo los siguientes acrónimos:

- **DMD:** Depósito a menor distancia
- **DMDP:** Depósito a menor distancia (vendedores ordenados por promedio de distancia)
- **VMD:** Vendedor a menor distancia
- **ARDD:** Asignación sujeta a ratio distancia-demanda

4.1. Metodología

En lo que respecta a entorno de experimentación, se usó un equipo que cuenta con un procesador Intel Core i3-9100F, 16 GB de memoria RAM DDR4 y un HDD de 1 TB de almacenamiento. En cuanto al sistema operativo, se usó Linux Mint 22 (Ubuntu 24.04). El lenguaje utilizado fue C++ (v13.3.0) con el compilador g++ con soporte de C++17 (probado con GCC ≥ 9). Es importante señalar que el archivo Makefile no fue testeado en entornos nativos de Windows.

4.2. Experimento 1: Comparación de tiempos de ejecución entre las heurísticas constructivas

Heurística	Corridas	Tiempo medio	Desvío estándar	Costo
DMD	20	1.11 ms	0.091 ms	788.2
DMDP	20	2.70 ms	0.036 ms	1770.3
VMD	20	461.60 ms	13.56 ms	850.6
ARDD	5	59.77 s	0.256 s	1417

Cuadro 1: Comparación de heurísticas constructivas sobre la instancia real ($m = 310$, $n = 1100$).

Como se puede observar en el cuadro, la primera heurística (depósito a menor distancia) no sólo probó ser la más rápida sino que también resultó ser la más eficiente para esta instancia real.

4.3. Experimento 2: Impacto de la heurística constructiva elegida para los operadores de búsqueda locales

Se busco comprender cuán influyente resulta en el desempeño de los operadores de búsqueda local la elección de la heurística constructiva con la que estos algoritmos obtienen una solución inicial factible. El experimento se llevó a cabo sobre los operadores `swap` y `relocate`. Además fueron empleadas las siguientes heurísticas analizadas en el experimento previo.

Para cada se decide generar una solución sobre la cual se ejecuta búsqueda local usando el operador **swap**. Tras analizar los resultados se repite el procedimiento pero usando el operador **relocate**. Cabe aclarar que para este experimento fue llevado a cabo sobre la misma instancia (`instances/real/real_instance`).

Cuadro 2: Impacto de la heurística constructiva sobre el desempeño de los operadores de búsqueda local. Cabe aclarar que no hicimos uso de la heurística constructiva que toma como criterio los ratios debido a que tuvimos complicaciones con los tiempos de ejecución.

HC	Operador	Mejora relativa (%)	Costo final
DMD	swap	7.47	729.3
DMD	relocate	0	788.2
DMDP	swap	8.87	775.1
DMDP	relocate	0	850.6
VMD	swap	30	1237
VMD	relocate	57	757

Definimos a la mejora relativa (m) como:

$$m = \frac{c_{\text{inicial}} - c_{\text{final}}}{c_{\text{inicial}}} \times 100$$

4.4. Experimento 3: Tuning de α en GRASP

Cuadro 3: Sensibilidad del parámetro α de GRASP sobre instancias de distinto tamaño (20 corridas por combinación). Para este experimento se utilizaron las instancias **a05100**, **a10200** y **a20200**, correspondientes a la carpeta **gap_a**.

Muestra	α	Corridas	Tiempo medio	Desvío estándar	Costo
Chica ($m = 5, n = 100$)	0	20	12.46 ms	4.03 ms	—
	0.25	20	15.84 ms	4.93 ms	1698
	0.5	20	15.77 ms	4.77 ms	1698
	0.75	20	14.30 ms	4.99 ms	1698
	1	20	15.03 ms	5.40 ms	1698
Mediana ($m = 10, n = 200$)	0	20	40.33 ms	4.96 ms	2624
	0.25	20	37.79 ms	4.12 ms	2626
	0.5	20	37.12 ms	3.86 ms	2626
	0.75	20	39.24 ms	5.67 ms	2623
	1	20	37.49 ms	3.87 ms	2623
Grande ($m = 20, n = 200$)	0	20	34.79 ms	3.76 ms	2343
	0.25	20	34.52 ms	5.23 ms	2348
	0.5	20	33.21 ms	4.15 ms	2346
	0.75	20	31.31 ms	3.09 ms	2343
	1	20	33.30 ms	3.10 ms	2343

5. Uso de herramientas de AI

Para el desarrollo del trabajo se utilizaron herramientas de inteligencia artificial, como Claude[1], con tres fines principales: la redacción de los pseudocódigos correspondientes a los algoritmos implementados, la consulta de aspectos relacionados con la complejidad temporal y espacial de dichos algoritmos, y, en las etapas iniciales del proyecto, la generación de los archivos .cpp y .h necesarios para la lectura de archivos.

6. Conclusiones

Los resultados muestran que, sobre la instancia real, la heurística más simple (DMD) fue a la vez la más rápida (1.11 ms) y la de mejor costo (788.2), superando a criterios más sofisticados como VMD y ARDD — al contrario de lo que se suele pensar, la complejidad adicional no se tradujo en mejores soluciones, pero sí en tiempos de ejecución mucho mayores.

El segundo experimento confirmó esta tendencia: aunque VMD mostró las mayores mejoras relativas con búsqueda local (30 % con swap, 57 % con relocate), partía de una solución mucho peor, y el mejor resultado final lo obtuvo igualmente DMD + swap (729.3). Es decir, partir de una buena solución constructiva resulta más efectivo que confiar en que la búsqueda local compense una mala solución inicial.

En cuanto a GRASP, el parámetro α no mostró un efecto claro ni monótono sobre el costo ni el tiempo: en la muestra chica el costo fue invariante para todo α , y en las muestras mediana y grande las diferencias entre valores extremos e intermedios fueron mínimas, sin que ningún α dominara consistentemente.

En conjunto, para el caso de estudio analizado, la combinación de una heurística constructiva simple con un operador de búsqueda local liviano (DMD + swap) ofreció el mejor balance entre calidad y costo computacional, por encima de la metaheurística GRASP.

En pos de guiar futuros estudios, resulta preciso evaluar estos algoritmos con más instancias reales y experimentar en una gama más amplia de dispositivos y sistemas operativos. Aún cuando las diferencias son muy significativas y se puede confiar en su replicabilidad, es de sumo interés ver cómo se pueden acrecentar o decrementar dichos márgenes dependiendo del dispositivo.

Referencias

- [1] Anthropic. *Conversación en Claude sobre el desarrollo del pseudocódigo y la complejidad de la metaheurística GRASP*. Disponible en: [link-chat](#)